

Paso 12 - Sobrecarga | Ejercicios

Dos métodos pueden llamarse igual si **cambian los parámetros** (en cantidad o en tipo). El tipo de retorno no cuenta.

Esta hoja agrega métodos a `Geometria` (B12.1 y B12.2) y a `Notas` (B12.3), así que **copia esas dos clases a tu carpeta de trabajo** antes de empezar: `Geometria.java` está en [referencia/](#) y `Notas.java` en [../paso10/referencia/Notas.java](#). El B12.4 y el B12.5 no necesitan nada.

► **Plantillas:** [plantillas/](#) — un `.java` por ejercicio, con el enunciado, la salida esperada y los `// TODO` donde va tu código. Si las usas, el archivo ya viene creado (`B12_1_Perimetro.java`) y **ese** nombre manda sobre el que diga el enunciado.

✓ **Solucionario:** [soluciones/](#) — el código de cada ejercicio, la salida real y por qué se resuelve así. Ábrelo **después** de intentarlo.

Cómo se lee cada ejercicio: el título dice qué hay que lograr; el bloque de abajo es la **salida esperada**, o sea lo que tiene que aparecer en tu pantalla (en los programas que preguntan cosas, lo que va después de la pregunta es lo que **escribe el usuario**); las citas como esta son pistas. Si no sabes por dónde partir, abre la plantilla: los `// TODO` numerados son los pasos, en orden. Compila y ejecuta después de cada uno.

B12.1 - `perimetro`, igual que `area`

Qué practicas: escribir varias versiones de un mismo método, distinguidas solo por la cantidad de parámetros.

Agrega a `Geometria` tres versiones de `perimetro`:

Figura	Parámetros	Fórmula
Círculo	<code>radio</code>	<code>2 * Math.PI * radio</code>
Rectángulo	<code>base</code> , <code>altura</code>	<code>2 * (base + altura)</code>
Triángulo	<code>a</code> , <code>b</code> , <code>c</code>	<code>a + b + c</code>

```
Circulo de radio 2:      area  12.57  perimetro  12.57
Rectangulo de 3 x 4:     area  12.00  perimetro  14.00
Triangulo de lados 3-4-5: area   6.00  perimetro  12.00
```

Para que las columnas queden alineadas, cada línea es un solo `printf` con el molde `"%-26sarea %6.2f perimetro %6.2f\n"` (sin espacio entre el `%-26s` y `area`: el relleno del `%-26s` ya lo pone).

El círculo de radio 2 tiene el área y el perímetro iguales. No es un error tuyo: pasa solo con el radio 2.

B12.2 - Validar el triángulo

Qué prácticas: por qué validar antes de calcular evita un resultado inválido (`NaN`) que después nadie detecta.

Agrega:

```
public static boolean esTriangulo(double a, double b, double c)
```

Un triángulo existe si **cada** lado es menor que la suma de los otros dos. Úsalo en el `main` antes de pedir el área:

```
Lados 3, 4, 5:  area 6.00
Lados 1, 1, 5:  ese triangulo no existe
```

Sin esta validación, `Geometria.area(1, 1, 5)` devuelve `NaN`. Compruébalo antes de arreglarlo: es el ejemplo más claro de por qué el paso 11 va antes que este.

B12.3 - El promedio ponderado

Qué prácticas: sobrecargar un método agregándole un parámetro, y recorrer dos arreglos en paralelo con el mismo índice.

Agrega a `Notas` una **segunda** versión de `promedio`:

```
public static double promedio(double[] notas, double[] pesos)
```

Multiplica cada nota por su peso y suma todo. Con notas `{5.0, 6.0, 4.0}` y pesos `{0.3, 0.3, 0.4}` tiene que dar **4.9**.

```
Promedio simple:    5.0
Promedio ponderado: 4.9
```

Ahora `Notas.promedio` tiene dos formas, y Java elige según cuántos arreglos le pases.

¿Qué pasa si los pesos no suman 1.0? ¿Y si el arreglo de pesos tiene menos elementos que el de notas? Decide qué debería hacer el método y déjalo anotado en un comentario: en la demo 10 eso se resuelve lanzando una excepción.

B12.4 - `sumar` para todo

Qué prácticas: entender exactamente cómo elige Java entre varias versiones sobrecargadas cuando el dato no calza exacto con ninguna.

En una clase nueva `Calculos`, escribe cuatro versiones de `sumar`:

```
public static int sumar(int a, int b)
public static double sumar(double a, double b)
public static int sumar(int a, int b, int c)
public static int sumar(int[] numeros)
```

Pruébalas todas y anota **cuál** eligió Java en cada caso:

```
Calculos.sumar(2, 3);
Calculos.sumar(2.5, 3.5);
Calculos.sumar(1, 2, 3);
Calculos.sumar(new int[] {1, 2, 3, 4});
Calculos.sumar(2, 3.5);           // esta es la interesante
```

La última no calza exacto con ninguna: ¿cuál usó y por qué? ¿Qué pasa si borras la versión de `double`?

B12.5 - Desafío: la tarjeta de saludo

Qué prácticas: encadenar versiones sobrecargadas para no repetir código, y comparar esta solución con la de Python.

En un archivo nuevo, escribe tres versiones de un mismo método que hagan lo mismo con más o menos datos, encadenándose una a la otra igual que el `linea()` del bloque 12.2 de la guía:

```
static void tarjeta(String nombre)
static void tarjeta(String nombre, String mensaje)
static void tarjeta(String nombre, String mensaje, char borde)
```

- La de un parámetro usa el mensaje `"Feliz cumpleaños"` y el borde `*`.

- La de dos usa el borde `*`.
- **Solo la de tres** dibuja algo: las otras dos la llaman.

```
*****  
* Feliz cumpleaños *  
* Ana *  
*****
```

Después escribe en un comentario cómo se haría lo mismo en Python, y cuántas funciones necesitarías.

Cuando termines, sigue con el [paso 13](#).